Demo Script, Viva Q&A and Handover Checklist

Disaster Communication and Resource Management System

1. Before you start (10 minutes early)

1. Power the **node** from a power bank. Plug the **gateway** into the laptop (USB).
2. On the laptop start the control station: `cd server`, then run the jar with the cloud settings (see README, "Cloud storage").
3. Open the dashboard `http://localhost:8000` and sign in as an officer.
4. Open `https://disaster-comm.onrender.com/cloud.html` in a second tab **now**. The free cloud sleeps and takes about a minute to wake.
5. Check the dashboard: node shows **online**, link **Wi-Fi**, and the Cloud sync box says **Synced**.
6. Have your phone ready with the Telegram bot chat open.
7. Hand-held lighter or a small cup of water ready for the flame and rain sensors.

2. The 8-minute demo

Step	Do	Say
1	Show the dashboard map and node table	"One sensor node reports water level, rain, flame and temperature every 3 seconds. The control room sees it live."
2	Hold a lighter near the flame sensor	"Flame detected. An incident opens automatically and suggests resources: fire engine and ambulance."
3	Show the phone	"The responsible officer is messaged on Telegram with severity, node, reading and a link."
4	Acknowledge the incident, allocate a fire engine, then resolve	"Every action is logged with the officer's name."
5	Run the Wi-Fi block script (separate terminal: `sudo scripts/node-wifi-outage.sh on`)	"Now I cut the node's Wi-Fi to simulate a disaster knocking out the network."
6	Wait 10 seconds; point at link = **LoRa** and the signal strength	"The node switched to LoRa by itself. The gateway received it. No data is lost."
7	Unblock (`... off`). Show readings returning to Wi-Fi	"When Wi-Fi returns, anything stored during the outage is replayed with its true time and the server drops duplicates."
8	Switch to the cloud tab	"Everything is also copied to a hosted cloud (Render with a Postgres database). If the Internet is down it queues and catches up later."
9	Pour a few drops on the rain plate, or show the thresholds dialog	"Thresholds are editable, so officers tune the alert rules without reflashing."

If something fails live: say "this is the fallback working as designed", switch to the cloud page or the screenshots in `docs/`, and carry on.

3. Viva questions and answers

What problem does it solve? During floods and fires the network is often the first thing to fail. This system keeps sending sensor data over a second radio path (LoRa) and tells officers instantly, so response is not delayed by a dead network.

Why ESP32? Built-in Wi-Fi, low cost, enough pins, runs on a power bank, and supports LoRa modules over SPI.

Why LoRa as the backup? Long range (kilometres), very low power, works without any infrastructure. It sends small packets, which is all a sensor reading needs.

How does the node decide to switch? It tries the Wi-Fi HTTP post to the server first. If that fails (no Wi-Fi or server unreachable) it sends the same reading over LoRa and also saves it in flash.

How are readings not lost or duplicated? Each reading has a node ID, boot number and sequence number. Unsent ones are stored in flash (up to 200) and replayed with their age. The server and cloud store each (node, boot, seq) only once.

How does the gateway reach the server with no Wi-Fi? A small bridge script reads the gateway over USB and posts to the server on the laptop, so the last hop needs no Wi-Fi.

What sensors and how do they work? HC-SR04 ultrasonic: time of the echo gives distance, so water level = mount height minus distance. DS18B20: digital temperature over one wire. Flame sensor: infrared photodiode with a digital output, LOW means flame. Rain plate: resistance drops when wet, read as an analog value.

Why a voltage divider on the sonar echo pin? The sensor outputs 5 V but ESP32 pins tolerate 3.3 V. A 2.2 k and 4.7 k divider lowers it safely.

Where do alert rules run? On the server. Flood level at or above 50 cm, flame, heat at or above 50 C, rain at or above 80. They are editable in the dashboard. A limit: the node itself does not decide, so alerts need the data to arrive first.

What happens when an incident opens? One incident per node and hazard until resolved. Suggested resources are shown, the responsible officer is messaged on Telegram, and officers acknowledge, allocate, assign and resolve it. Everything is audit-logged.

How is the system secured? Officer passwords are hashed with PBKDF2 and a random salt. Changing data needs login. The cloud accepts uploads only with an API key and never receives passwords. Secrets are kept out of git.

What database is used and why? H2 (a file database) on the control station for simplicity. Postgres (Neon) in the cloud for durability. Same Spring Boot code runs on both.

What is the "cloud" and how does sync work? The same Spring Boot app hosted on Render, with a Neon Postgres database. The station uploads batches of up to 500 readings every 10 seconds plus a snapshot, and moves its bookmark forward only after the cloud confirms. If the Internet drops, data stays queued.

What if Telegram or the Internet is down? Messages queue and retry for up to 24 hours. The dashboard sound and banner still work locally.

What are the limits? 433 MHz is for the demo; the licence-free band in India is 865-867 MHz. Smoke sensor is not fitted. One station per cloud. Telegram queue is in memory. Sensors need calibration on site.

What would you add next? Alert decisions on the node, an installable phone app, solar and battery power, more nodes with a mesh, an 865 MHz radio, and a login on the cloud page.

How did you test it? Nine automated tests, plus hardware tests: Wi-Fi blocked (32 of 32 readings arrived over LoRa), server down (31 of 31 replayed with true times), full Wi-Fi outage through the USB bridge, and cloud down then recovered.

4. Handover checklist (for the person receiving the project)

- [ ] GitHub repo: https://github.com/vanshdev0101/disaster-comm (public)
- [ ] Hardware: node ESP32, gateway ESP32, 2 LoRa modules, sensors, cables, power bank
- [ ] Both boards flashed (README, "Firmware"); `firmware/include/secrets.h` has the Wi-Fi name, password and the laptop's IP
- [ ] Laptop has JDK 21; server runs with `mvnw spring-boot:run`
- [ ] Telegram bot token in `server/config/application.properties`, officer account linked
- [ ] Cloud: https://disaster-comm.onrender.com/cloud.html and the receiver key (kept private)
- [ ] Officer login and admin password written down somewhere safe
- [ ] Wiring diagrams in `docs/`; project explainer `docs/project-explained.pdf`
- [ ] Practised the demo once start to finish
- [ ] Windows laptop: README "Windows" steps tried once before the day

5. Quick troubleshooting

Problem	Fix
Node offline	Check power bank, Wi-Fi name and password in `secrets.h`, laptop IP unchanged
Gateway keeps restarting	Weak USB cable or port; swap to a good cable
No LoRa data	Antennas on, both boards on the same frequency, boards within a few metres
Telegram silent	Needs Internet; re-check token and press Test message
Cloud page slow	Free hosting was asleep; wait a minute
Wrong board port	Ports swap between runs; identify by MAC address